

Einfache Graphenalgorithmen

Ausarbeitung zum Vortrag

Proseminar Kombinatorik

Sommersemester 2026

Lennart Meier

lennart.meier05@gmail.com

26.05.2026

Inhaltsverzeichnis

1	Einleitung	1
2	Grundbegriffe/Wiederholung Graphen	2
3	Graphendurchmusterung	3
4	Tiefensuche	5
5	Breitensuche	6
6	Bipartite Graphen	7
7	Azyklische Digraphen	8
8	Fazit	9
	Literatur	10

1 Einleitung

Diese Ausarbeitung behandelt einfache Graphenalgorithmien und basiert auf Kapitel 7 aus [1]. Graphen gehören zu den wichtigsten Strukturen der Mathematik und Informatik. Viele Probleme lassen sich mithilfe von Graphen darstellen. Beispiele hierfür sind:

- Straßennetze, Bahnnetze,
- soziale Netzwerke,
- Kommunikationsnetze,
- Projektplanung.

Ein zentrales Problem besteht darin, einen Graphen systematisch zu durchsuchen und Informationen über seine Struktur zu gewinnen. Dabei interessiert insbesondere:

- welche Knoten erreichbar sind
- wie kürzeste Wege bestimmt werden können
- ob ein Graph bipartit ist
- ob ein gerichteter Graph Kreise enthält

Im Folgenden werden zunächst allgemeine Verfahren zur Graphendurchmusterung vorgestellt. Anschließend werden spezielle Algorithmen wie Tiefensuche und Breitensuche untersucht. Danach werden bipartite Graphen sowie azyklische Digraphen betrachtet.

2 Grundbegriffe/Wiederholung Graphen

Definition 2.1. *Ein Graph ist ein Paar*

$$G = (V, E),$$

wobei

- V eine endliche Menge von Knoten ist,
- E eine Menge von Kanten ist.

Bei ungerichteten Graphen besteht jede Kante aus einer ungeordneten Menge

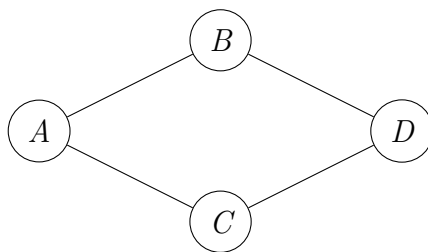
$$\{u, v\}.$$

Bei gerichteten Graphen besitzen die Kanten dagegen eine Richtung und werden als geordnete Paare

$$(u, v)$$

geschrieben.

Beispiel 2.2. *Der folgende Graph besitzt vier Knoten und vier Kanten:*



Es gilt:

$$V = \{A, B, C, D\},$$

$$E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}\}.$$

Definition 2.3. *Ein Weg ist eine Folge von Knoten, bei der aufeinanderfolgende Knoten durch eine Kante verbunden sind.*

Ein Kreis ist ein geschlossener Weg.

Ein Graph heißt zusammenhängend, wenn zwischen je zwei Knoten ein Weg existiert.

3 Graphendurchmusterung

Das grundlegende Verfahren zur Untersuchung eines Graphen ist die sogenannte Graphendurchmusterung.

Algorithmus 3.1 (Graphendurchmusterung). *Eingabe: ein Graph G , ein Knoten $r \in V(G)$*

Ausgabe: die Menge $R \subseteq V(G)$ der von r aus erreichbaren Knoten und eine Menge $F \subseteq E(G)$, sodass (R, F) ein Baum beziehungsweise eine Arboreszenz mit Wurzel r ist.

```

 $R \leftarrow \{r\}, \quad Q \leftarrow \{r\}, \quad F \leftarrow \emptyset$ 
while  $Q \neq \emptyset$  do
    Wähle ein  $v \in Q$ 
    if  $\exists e = (v, w) \in \delta_G^+(v)$ 
        mit  $w \in V(G) \setminus R$ 
    then  $R \leftarrow R \cup \{w\}$ 
         $Q \leftarrow Q \cup \{w\}$ 
         $F \leftarrow F \cup \{e\}$ 
    else  $Q \leftarrow Q \setminus \{v\}$ 

```

Satz 3.2. *Der Algorithmus zur Graphendurchmusterung arbeitet korrekt und kann mit Laufzeit*

$$O(n + m)$$

implementiert werden, wobei

$$n = |V(G)|, \quad m = |E(G)|.$$

Beweis. Zu Beginn besteht (R, F) lediglich aus dem Startknoten r . Somit ist (R, F) ein Baum beziehungsweise eine Arboreszenz mit Wurzel r .

Im Verlauf des Algorithmus wird genau ein neuer Knoten w zusammen mit einer Kante e hinzugefügt, wobei der andere Endknoten der Kante bereits in R liegt. Dadurch bleibt die Baumstruktur erhalten. Angenommen, am Ende des Algorithmus gäbe es einen von r aus erreichbaren Knoten w , der nicht in R enthalten ist. Dann existiert ein Weg von r nach w . Auf diesem Weg muss es eine Kante geben, die einen Knoten aus R mit einem Knoten außerhalb von R verbindet. Eine solche Kante hätte der Algorithmus jedoch betrachtet und den neuen Knoten hinzugefügt. Dies ist ein Widerspruch. Für die Laufzeit betrachtet man jede Kante höchstens zweimal. Jeder Knoten wird höchstens einmal eingefügt und entfernt. Daher ergibt sich insgesamt eine lineare Laufzeit

$$O(n + m).$$

□

Korollar 3.3. *Die Zusammenhangskomponenten eines ungerichteten Graphen können in linearer Zeit bestimmt werden.*

Beweis. Man startet den Algorithmus in einem beliebigen Knoten. Falls nicht alle Knoten erreicht werden, bildet die Menge der erreichten Knoten eine Zusammenhangskomponente. Anschließend wiederholt man den Algorithmus auf den verbleibenden Knoten. □

4 Tiefensuche

Eine wichtige Variante der Graphendurchmusterung ist die Tiefensuche.

Definition 4.1. *Wählt man in der Menge Q stets den zuletzt hinzugefügten Knoten aus, so erhält man die Tiefensuche beziehungsweise DFS (Depth First Search).*

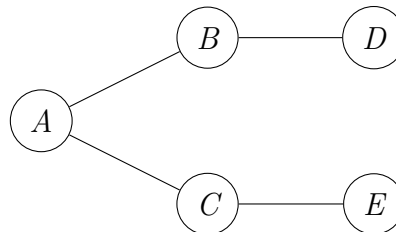
Die Tiefensuche arbeitet nach dem Prinzip:

Gehe erst möglichst tief in den Graphen, bevor man zurück geht.

Dafür verwendet DFS einen Stack.

Beispiel 4.2. *Eine mögliche DFS-Reihenfolge im folgenden Graphen lautet:*

$A, B, D, C, E.$



DFS wird unter anderem verwendet für:

- Zusammenhangsprobleme,
- Kreissuche,
- topologische Sortierung,
- Komponentenzerlegungen.

5 Breitensuche

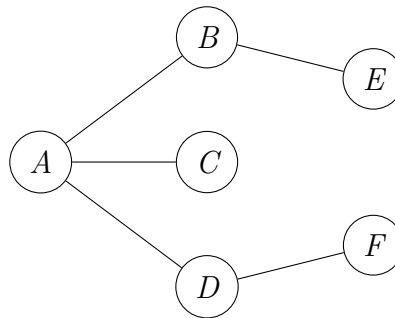
Definition 5.1. *Wählt man in der Menge Q stets den zuerst hinzugefügten Knoten aus, so erhält man die Breitensuche beziehungsweise BFS (Breadth First Search).*

BFS durchsucht den Graphen schichtweise. Zunächst werden alle Nachbarn des Startknotens besucht, danach alle Nachbarn dieser Knoten usw.

Hierfür wird eine Queue verwendet.

Beispiel 5.2. *Im folgenden Graphen ergeben sich die BFS-Ebenen*

$$\{A\}, \{B, C, D\}, \{E, F\}.$$



Für zwei Knoten v, w bezeichnet

$$dist(v, w)$$

die Länge eines kürzesten Weges von v nach w .

Satz 5.3. *Jeder BFS-Baum enthält einen kürzesten Weg von einem Knoten r zu allen anderen von r aus erreichbaren Knoten.*

Beweis. Die Breitensuche arbeitet schichtweise. Zunächst werden alle Knoten mit Abstand 1 besucht, danach alle Knoten mit Abstand 2 usw.

Ein Knoten wird also erstmals genau dann erreicht, wenn ein kürzester Weg zu ihm gefunden wurde. Daher enthält der BFS-Baum nur kürzeste Wege. $\square$

Satz 5.4. *Die Werte*

$$\text{dist}(r, v)$$

können für alle Knoten v in linearer Zeit bestimmt werden.

Beweis. Jeder Knoten wird höchstens einmal besucht und jede Kante höchstens zweimal betrachtet. Daher beträgt die Laufzeit insgesamt

$$O(n + m).$$

$\square$

6 Bipartite Graphen

Definition 6.1. *Eine Bipartition eines ungerichteten Graphen G besteht aus zwei disjunkten Knotenmengen A und B , deren Vereinigung $V(G)$ ist, sodass jede Kante genau einen Endknoten in A besitzt. Ein Graph heißt bipartit, falls er eine Bipartition besitzt.*

Definition 6.2. *Ein ungerader Kreis ist ein Kreis ungerader Länge.*

Satz 6.3 (König). *Ein ungerichteter Graph ist genau dann bipartit, wenn er keinen ungeraden Kreis enthält.*

Beweis. Sei zunächst G bipartit mit Bipartition

$$V(G) = A \cup B.$$

Durchläuft man einen Kreis, so wechselt man nach jeder Kante zwischen den Mengen A und B . Daher muss jeder Kreis gerade Länge besitzen. Somit existiert kein ungerader Kreis. Nun sei umgekehrt G ein Graph ohne

ungeraden Kreis. Man führt eine Breitensuche mit Startknoten r durch und definiert:

$$A := \{v \in V(G) \mid \text{dist}(r, v) \text{ gerade}\},$$

$$B := V(G) \setminus A.$$

Angenommen, es gäbe eine Kante innerhalb einer der beiden Mengen. Dann entstünde zusammen mit den BFS-Wegen ein ungerader Kreis. Dies widerspricht der Voraussetzung. Folglich ist der Graph bipartit. $\square$

7 Azyklische Digraphen

Definition 7.1. *Ein Digraph heißt azyklisch, falls er keinen gerichteten Kreis enthält.*

Definition 7.2. *Sei*

$$V(G) = \{v_1, \dots, v_n\}.$$

Eine topologische Ordnung ist eine Ordnung der Knoten, sodass

$$i < j$$

für jede Kante

$$(v_i, v_j) \in E(G)$$

gilt.

Lemma 7.3. *Ist G ein Digraph mit*

$$\delta^+(v) \neq \emptyset$$

für alle $v \in V(G)$, so kann man in

$$O(|V(G)|)$$

Zeit einen Kreis in G finden.

Beweis. Man startet in einem beliebigen Knoten und folgt jeweils einer ausgehenden Kante. Da der Graph nur endlich viele Knoten besitzt, muss irgendwann ein Knoten erneut besucht werden. Dadurch entsteht ein Kreis. $\square$

Satz 7.4. *Ein Digraph besitzt genau dann eine topologische Ordnung, wenn er azyklisch ist.*

Beweis. Besitzt ein Digraph eine topologische Ordnung, so kann kein gerichteter Kreis existieren, da entlang jeder Kante die Nummerierung streng anwächst. Nun sei G azyklisch. Dann existiert mindestens ein Knoten ohne ausgehende Kante. Entfernt man diesen Knoten, bleibt erneut ein azyklischer Graph zurück. Durch wiederholtes Entfernen solcher Knoten entsteht schließlich eine Reihenfolge

$$v_1, \dots, v_n,$$

die eine topologische Ordnung bildet. $\square$

8 Fazit

In dieser Ausarbeitung wurden einfache Graphenalgorithmien untersucht. Die Graphendurchmusterung bildet die Grundlage vieler weiterer Verfahren. Besonders wichtig sind die beiden Varianten Tiefensuche und Breitensuche. Die Breitensuche ermöglicht zusätzlich die Berechnung kürzester Wege in ungewichteten Graphen. Außerdem liefert sie wichtige Anwendungen beim Test auf Bipartitheit. Außerdem wurden azyklische Digraphen betrachtet. Für diese Graphen existieren topologische Ordnungen, die insbesondere bei Abhängigkeitsproblemen verwendet werden.

Literatur

- [1] Stefan Hougardy und Jens Vygen. *Algorithmische Mathematik*. 2., korrigierte und erweiterte Auflage. Berlin: Springer Spektrum, 2018.